TRANSLATE APP

Backend API Documentation

Endpoints, request bodies, response types, workflows, deployment contracts, and implementation caveats

System: NestJS 11 / TypeORM / MySQL / Firebase Admin / Google Places / OCR and speech AI services

Source baseline: local repository implementation inspected on 19 August 2026

API version: Swagger document 1.0; package version 0.0.1

This is an implementation-derived reference. Where code and Swagger metadata disagree, this document calls out the actual runtime behavior.

1. Executive overview

The backend supports a multilingual travel expense application. It combines account authentication, localized reference data, receipt OCR, merchant recognition, expenses and statistics, live support chat, travel exploration, hourly nearby-place guidance, speech interpretation, notifications, subscription plans, payments, FAQs, and administrator analytics.

Primary integrations

Integration	Role	Backend contract
Firebase Authentication	Identity and bearer-token verification	ID tokens protect authenticated routes; refresh tokens are exchanged by /auth/refresh.
Firebase Realtime Database	Live support-chat delivery	REST persists SQL history; clients subscribe to RTDB for live messages.
Firebase Cloud Messaging	Push notifications	Device tokens are registered under /notifications/devices.
Google Places API (New)	Merchant matching and nearby discovery	Server-side GOOGLE_MAPS_API_KEY; clients never receive the key.
OCR service	Receipt extraction	Multipart image/PDF forwarded through OCR_SERVICE_URL.
Translator / speech service	Text/JSON translation, STT and TTS	TRANSLATOR_SERVICE_URL; expected /ai/* routes.
Payment providers	Konnect and Moyasar checkout/webhooks	Provider selected by user/country configuration in payment service.

Base URL and documentation

- No global API prefix is configured. A deployed example is **http://54.37.227.247:3003**; production should use an HTTPS domain.
- Swagger UI is mounted at **/api-docs**.
- GET **/** is a public liveness-style route returning the text "this is the main page of Translate App backend services !" inside the standard envelope.
- Static files are mounted at **/files**, including profile images and interpreter audio.
- The server listens on **PORT**, defaulting to 3000. CORS is globally enabled.

2. Cross-cutting API contract

Authentication

Authenticated requests use a Firebase ID token:

```
Authorization: Bearer <firebase-id-token>
```

The FirebaseAuthGuard verifies the token with Firebase Admin, synchronizes or loads the SQL user, and attaches both the SQL user and decoded Firebase identity to the request. Missing tokens return 401 "Missing bearer token"; invalid or expired tokens return 401 "Invalid or expired token". Admin routes additionally require role=admin.

Success envelope

```
{
  "status_code": 200,
  "title": "Loaded successfully",
  "body": "Loaded successfully",
  "data": { ...endpoint-specific payload... }
}
```

All non-204 controller responses are wrapped. Default messages are method-based: GET "Loaded successfully", POST "Created successfully", PATCH/PUT "Updated successfully", DELETE "Deleted successfully". The message may be translated into the authenticated user's selected language.

Error envelope

```
{
  "status_code": 400,
  "title": "Validation or domain error message",
  "body": "Validation or domain error message",
  "data": null
}
```

Validation and status notes

- 204 routes return no body and bypass the success envelope.
- DTO constraints are documented below, but **main.ts does not install a global ValidationPipe**. Validation is guaranteed only where a controller or route explicitly installs one. This is a current implementation caveat.
- The HTTP exception filter wraps Nest HttpException instances. Unexpected non-HTTP exceptions may use Nest default handling.
- Some GET data is translated using the user language while IDs, dates, codes, amounts, and technical fields are deliberately excluded.
- Common failures: 400 validation/bad file; 401 authentication; 403 admin/subscription; 404 missing owned resource; 409 duplicate record; 429 throttled; 502 upstream OCR/translation/speech/Places failure; 503 missing integration configuration.

Reusable resource models

Model	Key fields
User	id UUID; role user|admin; firebaseUid; email; displayName; photoURL; provider; isActive; country/currency/language relations; subscriptionStatus; trialEndsAt; subscriptionEndsAt; timestamps
Expense	id; amount; description?; date; category/categoryId; currency/currencyId; shop?; Google place ID/address/coordinates/maps URI/confidence; userId; timestamps
Receipt	id; userId?; documentType; language; country; merchant; address; phone; invoiceNumber; date/time; currency; subtotal/tax/total; paymentMethod; rawText; items; file metadata; timestamps
Notification	id; userId; title; body; data; source manual|event; read; pushSent; createdAt
Conversation / Message	support conversation counters and preview; message id, senderId, senderRole user|admin, body, read, createdAt
PaymentPlan / Payment	plan pricing and duration; payment user/plan/amount/currency/status/provider/reference/timestamps

3. Authentication and account lifecycle

POST /auth/register

Access	Public
Purpose	Create Firebase email/password identity and SQL user.
Request	JSON: { email: valid email, password: string min 6 }
Success data	{ user: User, idToken: string, refreshToken: string }

Notes: 201; duplicate or invalid credentials normally 400.

POST /auth/login

Access	Public
Purpose	Sign in with email/password and synchronize the SQL profile.
Request	Same as register.
Success data	{ user, idToken, refreshToken }

Notes: Use idToken as bearer token; persist refreshToken securely.

POST /auth/oauth

Access	Public
Purpose	Exchange a Google or Facebook provider token.
Request	JSON: { provider: "google.com"|"facebook.com", token: string }
Success data	{ user, idToken, refreshToken }

Notes: Provider redirect URI is currently hard-coded to http://localhost:3000 in the controller.

POST /auth/firebase-token

Access	Bearer
Purpose	Create a Firebase custom token containing the SQL role.
Request	No body.
Success data	{ customToken: string }

Notes: Useful for RTDB rules and live chat.

POST /auth/refresh

Access	Public
Purpose	Exchange a refresh token for a new Firebase session.
Request	JSON: { refreshToken: string }
Success data	Firebase refresh response containing a fresh ID token and refresh token.

Notes: Call after a 401 caused by expiry; redirect to login if no refresh token or refresh fails.

POST /auth/send-reset-mail

Access	Public; 3 requests / 15 min
Purpose	Send a four-digit password-reset OTP.
Request	JSON: { email: valid email }
Success data	Reset-mail acknowledgement.

Notes: Step 1 of 3.

POST /auth/validate-reset-mail

Access	Public; 5 requests / 5 min
Purpose	Validate OTP and issue resetPassToken.
Request	JSON: { email, otp: exactly 4 characters }
Success data	Object containing resetPassToken.

Notes: Step 2; preserve token client-side for step 3.

POST /auth/reset-password

Access	Public; 5 requests / 10 min
Purpose	Replace password after OTP validation.
Request	JSON: { email, resetPassToken: exactly 4 chars, newPassword: min 6 }
Success data	Reset acknowledgement.

Notes: Step 3; token is mandatory.

GET /users/me

Access	Bearer
Purpose	Load authenticated profile.
Request	None
Success data	User

Notes: Refetch after language/currency/country changes.

PATCH /users/me

Access	Bearer; multipart/form-data
Purpose	Update profile and optionally upload avatar.
Request	Fields: displayName?, currencyId UUID?, currentCountryId UUID?, languageId UUID?, image file?
Success data	Updated User

Notes: Image is stored under /files/profile-images; provider restrictions may apply.

DELETE /users/me

Access	Bearer
Purpose	Permanently delete own account after email confirmation.
Request	JSON: { confirmEmail: valid email }
Success data	Deletion result.

Notes: Use a destructive-action confirmation UI.

Password-reset scenario

```
POST /auth/send-reset-mail        { "email": "user@example.com" }
POST /auth/validate-reset-mail    { "email": "user@example.com", "otp": "1234" }
POST /auth/reset-password         { "email": "user@example.com", "resetPassToken": "5678", "newPassword": "newSecret" }
```

4. Reference data and localization

GET /currencies

Access	Optional bearer
Purpose	List currencies; names translated when user context exists.
Request	None
Success data	Currency[]

Notes: Currency: id, code ISO-3, name, isActive.

GET /currencies/convert

Access	Public
Purpose	Convert using database currency IDs.
Request	Query: amount >= 0.01, fromCurrencyId UUID, toCurrencyId UUID
Success data	Conversion result

Notes: IDs are not ISO codes.

GET /currencies/:id

Access	Optional bearer
Purpose	Get currency.
Request	Path id
Success data	Currency

POST /currencies

Access	Admin bearer
Purpose	Create currency.
Request	{ code: exactly 3 chars, name, isActive? }
Success data	Currency

PATCH /currencies/:id

Access	Admin bearer
Purpose	Update currency.
Request	Partial create body.
Success data	Currency

DELETE /currencies/:id

Access	Admin bearer
Purpose	Delete currency.
Request	Path id
Success data	Deletion result

GET /countries

Access	Optional bearer
Purpose	List translated countries.
Request	None
Success data	Country[]

Notes: Country fields include alpha2Code, alpha3Code, flag, region, capital, currencyId, isActive.

GET /countries/:id

Access	Optional bearer
Purpose	Get translated country.
Request	Path id
Success data	Country

POST /countries

Access	Admin bearer
Purpose	Create country.
Request	{ name, alpha2Code length 2, alpha3Code length 3, flag?, region?, capital?, currencyId UUID?, isActive? }
Success data	Country

PATCH /countries/:id

Access	Admin bearer
Purpose	Update country.
Request	Partial create body.
Success data	Country

DELETE /countries/:id

Access	Admin bearer
Purpose	Delete country.
Request	Path id
Success data	Deletion result

GET /languages

Access	Public
Purpose	List languages.
Request	None
Success data	Language[]

Notes: Language: id, code, name.

GET /languages/search?q=...

Access	Public
Purpose	Search languages.
Request	Query q
Success data	Language[]

GET /languages/:id

Access	Public
Purpose	Get language.
Request	Path id
Success data	Language

POST /languages

Access	Admin bearer
Purpose	Create language.
Request	{ code: 2-5 chars, name }
Success data	Language

PATCH /languages/:id

Access	Admin bearer
Purpose	Update language.
Request	Partial create body.
Success data	Language

DELETE /languages/:id

Access	Admin bearer
Purpose	Delete language.
Request	Path id
Success data	Deletion result

POST /translation/text

Access	Bearer
Purpose	Translate text; target defaults to user language.
Request	{ text, target?, source? }
Success data	{ translation: string }

Notes: Upstream failure falls back to original text in TranslationService.

POST /translation/json

Access	Bearer in current implementation
Purpose	Translate JSON recursively.
Request	{ data: any JSON, target?, source?, ignoreKeys?: string[] }
Success data	{ data: translated JSON }

Notes: Although a method-level optional guard is present, the controller-level Firebase guard still applies.

POST /guest/translation/json

Access	Public
Purpose	Guest JSON translation without persistence.
Request	Same JSON translation body.
Success data	{ data }

Notes: Use this route for anonymous clients.

5. Expense categories and expenses

POST /expense-categories

Access	Bearer
Purpose	Create a user-owned category.
Request	{ name max 100, icon?, color?, description? max 500 }
Success data	ExpenseCategory

GET /expense-categories

Access	Bearer
Purpose	List own categories with translated display text.
Request	None
Success data	ExpenseCategory[]

GET /expense-categories/:id

Access	Bearer
Purpose	Get owned category.
Request	Path UUID
Success data	ExpenseCategory

Notes: Cross-user access behaves as not found.

PATCH /expense-categories/:id

Access	Bearer
Purpose	Update owned category.
Request	Partial create body.
Success data	ExpenseCategory

DELETE /expense-categories/:id

Access	Bearer
Purpose	Delete owned category.
Request	Path UUID
Success data	No content

Notes: 204.

POST /expenses

Access	Bearer
Purpose	Create expense.
Request	{ amount positive, description?, shop?, date ISO, categoryId UUID, currencyId?, googlePlaceId?, merchantAddress?, merchantLatitude?, merchantLongitude?, googleMapsUri?, merchantMatchConfidence 0-100? }
Success data	Expense

Notes: Currency defaults to user preference.

POST /expenses/from-receipt

Access	Bearer
Purpose	Create expense from an owned receipt.
Request	{ receiptId UUID, categoryId?, description?, amount?, date?, merchantName?, Google place fields? }
Success data	{ expense, convertedAmount, ...conversion metadata }

Notes: Receipt total/date/merchant supply defaults; returns home-currency conversion.

GET /expenses

Access	Bearer
Purpose	List expenses including today.
Request	Query: startDate ISO?, endDate ISO?, categoryId UUID?
Success data	Expense[]

Notes: Date range is inclusive according to service filtering.

GET /expenses/statistics

Access	Bearer
Purpose	Calculate total/count, category breakdown, and time series.
Request	Query: startDate?, endDate?, groupBy=day|month|year (default month)
Success data	{ total, count, byCategory, overTime, groupBy }

Notes: Declare before /:id route.

GET /expenses/:id

Access	Bearer
Purpose	Get owned expense.
Request	Path UUID
Success data	Expense

PATCH /expenses/:id

Access	Bearer
Purpose	Update owned expense.
Request	Partial create-expense body.
Success data	Expense

DELETE /expenses/:id

Access	Bearer
Purpose	Delete owned expense.
Request	Path UUID
Success data	No content

Notes: 204.

Create-expense example

```
POST /expenses
Authorization: Bearer <id-token>
Content-Type: application/json

{
  "amount": 42.5,
  "description": "Weekly groceries",
  "shop": "Carrefour Market Gabes",
  "date": "2026-08-19T10:30:00.000Z",
  "categoryId": "<category-uuid>",
  "currencyId": "<currency-uuid>",
  "googlePlaceId": "ChIJ...",
  "merchantLatitude": 33.8815,
  "merchantLongitude": 10.0982,
  "googleMapsUri": "https://maps.google.com/?cid=...",
  "merchantMatchConfidence": 91
}
```

6. Receipt OCR and merchant recognition

POST /receipts/extract

Access	Bearer; multipart; subscription may be enforced by module/service
Purpose	Extract and persist a receipt.
Request	multipart field image; jpg/jpeg/png/webp/pdf; max 10 MiB
Success data	Receipt with items and converted values where available.

Notes: Calls OCR service; ownership is assigned to authenticated user.

POST /guest/receipts/extract

Access	Public; 5/min
Purpose	Extract receipt without persistence.
Request	Same image constraints.
Success data	OCR extraction payload

Notes: Guest/demo route.

POST /receipts/merchant-match

Access	Bearer; 10/min
Purpose	Match OCR merchant to nearby Google place.
Request	{ merchantName 2-160, receiptAddress? max 300, latitude, longitude, languageCode? en|fr|ar }
Success data	Ranked merchant candidates with place IDs, map URIs, coordinates and confidence.

Notes: Requires Places API (New).

POST /receipts/receipt-recommendations/:receiptId

Access	Bearer
Purpose	Recommend expense category and description.
Request	Path receipt UUID
Success data	Recommendation object

Notes: Owned receipt required.

GET /receipts

Access	Bearer
Purpose	List owned receipts.
Request	Documented query: merchant?, page default 1, limit default 20
Success data	Receipt[]

Notes: Current controller does not bind @Query(), so filters/pagination are not passed to the service.

GET /receipts/:id

Access	Bearer
Purpose	Get owned translated receipt.
Request	Path UUID
Success data	Receipt

Notes: Amounts/technical fields are excluded from translation.

PATCH /receipts/:id

Access	Bearer
Purpose	Correct OCR result and items.
Request	Fields merchant/address/phone/invoiceNumber/date/time/currency/subtotal/tax/total/paymentMethod; items[] with id? and prices.
Success data	Receipt

Notes: Existing item id updates; missing id adds.

DELETE /receipts/:id

Access	Bearer
Purpose	Delete owned receipt.
Request	Path UUID
Success data	Deletion result

Normal scan-to-expense scenario

- 1. Capture/upload image and POST /receipts/extract.
- 2. Display extracted merchant, date, currency, total and line items for correction.
- 3. With device coordinates, POST /receipts/merchant-match and ask the user to confirm a candidate.
- 4. POST /receipts/receipt-recommendations/:receiptId for category/description assistance.
- 5. POST /expenses/from-receipt with overrides and confirmed Google place metadata.
- 6. Refetch expense history/statistics and expose googleMapsUri as a merchant link.

7. Explore and nearby guidance

POST /explore/nearby

Access	Bearer; 20/min
Purpose	Find nearby attractions, restaurants, cafes, museums, parks and shopping.
Request	{ latitude -90..90, longitude -180..180, radius 500..20000 default 10000, languageCode? en|fr|ar }
Success data	Categorized nearby place cards with distance and Google Maps data.

Notes: Coordinates come from the client browser/device.

POST /explore/check-in

Access	Bearer
Purpose	Choose a nearby place and create an in-app/FCM notification.
Request	{ latitude, longitude, accuracy? 0..10000, languageCode? en|fr|ar }
Success data	{ recommended: boolean, reason? } or recommendation and notification ID.

Notes: Current testing cooldown is one minute; same place two minutes. Increase before production.

POST /guest/explore/nearby

Access	Public; throttled
Purpose	Anonymous nearby search.
Request	NearbySearchDto
Success data	Nearby places

Notes: See guest controller for throttle settings.

GET /guest/explore/place/:placeId

Access	Public
Purpose	Load one Google place.
Request	Path placeId; query languageCode?
Success data	Place details

POST /guest/explore/place/:placeId/narration

Access	Public
Purpose	Generate place narration.
Request	Path placeId; JSON: { targetLanguage?: string }, default en
Success data	Narration payload

Notes: Used for guided discovery.

Hourly guide scenario

The frontend obtains a fresh browser position, posts it to /explore/check-in, then refreshes /notifications/me when recommended=true. The backend searches within approximately 3 km, chooses the nearest eligible new place, suppresses recent duplicates, persists the notification, and attempts FCM delivery. Web geolocation requires HTTPS except on localhost.

8. Interpreter

POST /interpreter/conversations

Access	Bearer
Purpose	Start a saved bilingual conversation.
Request	{ travelerLanguage 2-35, localLanguage 2-35, title? max 120 }
Success data	InterpreterConversation

GET /interpreter/conversations

Access	Bearer
Purpose	List own conversations newest first.
Request	None
Success data	InterpreterConversation[]

GET /interpreter/conversations/:id

Access	Bearer
Purpose	Load conversation and chronological turns.
Request	Path UUID
Success data	InterpreterConversation with turns[]

POST /interpreter/conversations/:id/turns

Access	Bearer; multipart/form-data
Purpose	Transcribe, translate, synthesize and persist one turn.
Request	audio file <=10 MiB plus speaker traveler|local, sourceLanguage, targetLanguage
Success data	InterpreterTurn

Notes: audio MIME must start with audio/. Calls /ai/speech-to-text, /ai/translate, /ai/text-to-speech.

DELETE /interpreter/conversations/:id

Access	Bearer
Purpose	Delete conversation and audio files.
Request	Path UUID
Success data	{ deleted: true }

POST /guest/interpreter/turn

Access	Public; throttled; multipart
Purpose	Process an unsaved interpreter turn.
Request	Same audio and turn fields.
Success data	Transcription, translation and synthesized audio response.

Notes: Guest/demo route.

Interpreter turn response type

```
{
  "id": "uuid",
  "conversationId": "uuid",
  "speaker": "traveler",
  "sourceLanguage": "en",
  "targetLanguage": "fr",
  "sourceText": "Where is the station?",
  "translatedText": "Ou est la gare ?",
  "sourceAudioUrl": "/files/interpreter/<uuid>.webm",
  "translatedAudioUrl": "/files/interpreter/<uuid>.mp3",
  "sourceMimeType": "audio/webm",
  "translatedMimeType": "audio/mpeg",
  "createdAt": "ISO timestamp"
}
```

9. Notifications and support chat

POST /notifications

Access	Bearer
Purpose	Create an in-app notification for self and optionally push.
Request	{ title, body, data?: string map, sendPush?: boolean default true }
Success data	Notification

Notes: Client-supplied userId is overridden with authenticated user ID.

POST /notifications/check-country

Access	Bearer
Purpose	Notify when detected country differs from declared country.
Request	{ detectedCountryCode: two-letter ISO code }
Success data	Country-check result and possibly Notification

GET /notifications/me

Access	Bearer
Purpose	List own notifications newest first.
Request	None
Success data	Notification[]

Notes: Titles/bodies may be translated.

GET /notifications/:id

Access	Bearer
Purpose	Get owned notification.
Request	Path UUID
Success data	Notification

PATCH /notifications/:id

Access	Bearer
Purpose	Update owned notification.
Request	Partial title/body/data/userId plus read?; sendPush excluded.
Success data	Notification

Notes: Service ownership protects target.

PATCH /notifications/:id/read

Access	Bearer
Purpose	Mark one read.
Request	Path UUID
Success data	Notification

PATCH /notifications/me/read-all

Access	Bearer
Purpose	Mark all own notifications read.
Request	None
Success data	{ updated: true }

DELETE /notifications/:id

Access	Bearer
Purpose	Delete owned notification.
Request	Path UUID
Success data	Removed Notification

POST /notifications/devices

Access	Bearer
Purpose	Register/update FCM device token.
Request	{ token, platform: web|ios|android }
Success data	DeviceToken

Notes: Authenticated user ID overrides body.

DELETE /notifications/devices/:token

Access	Bearer
Purpose	Unregister token at logout.
Request	Path token
Success data	{ removed: true }

Notes: URL-encode token when needed.

GET /chat/conversations/me

Access	Bearer
Purpose	Get or create own support conversation.
Request	None
Success data	Conversation

GET /chat/conversations/:id/messages

Access	Bearer owner or admin
Purpose	Paginate messages.
Request	Query before? ISO timestamp, limit? number
Success data	Message[]

Notes: Newest page first.

POST /chat/conversations/:id/messages

Access	Bearer owner or admin
Purpose	Persist and mirror a live message.
Request	{ body: 1-4000 chars }
Success data	Message

Notes: Clients receive live updates through RTDB.

PATCH /chat/conversations/:id/read

Access	Bearer owner or admin
Purpose	Mark conversation read for current role.
Request	None
Success data	null/void wrapped as data:null

GET /chat/conversations

Access	Admin bearer
Purpose	List all support conversations.
Request	None
Success data	Conversation[]

10. FAQs

GET /faqs/categories

Access	Public; optional bearer
Purpose	List active FAQ categories translated for user when present.
Request	None
Success data	FaqCategory[]

GET /faqs/categories/:categoryId

Access	Public; optional bearer
Purpose	Get category.
Request	Path UUID
Success data	FaqCategory

POST /faqs/categories

Access	Admin bearer
Purpose	Create category.
Request	{ name max 100, description? max 255, order? int, isActive? }
Success data	FaqCategory

Notes: 409 duplicate name.

PATCH /faqs/categories/:categoryId

Access	Admin bearer
Purpose	Update category.
Request	Partial create body.
Success data	FaqCategory

DELETE /faqs/categories/:categoryId

Access	Admin bearer
Purpose	Delete category; FAQs become uncategorized.
Request	Path UUID
Success data	No content

Notes: 204.

GET /faqs

Access	Public; optional bearer
Purpose	List FAQs, optionally by category.
Request	Query categoryId UUID?
Success data	Faq[]

GET /faqs/category/:categoryId

Access	Public; optional bearer
Purpose	List FAQs in category.
Request	Path UUID
Success data	Faq[]

GET /faqs/:id

Access	Public; optional bearer
Purpose	Get FAQ.
Request	Path UUID
Success data	Faq

POST /faqs

Access	Admin bearer
Purpose	Create FAQ.
Request	{ question max 255, answer, categoryId?, order? int, isActive? }
Success data	Faq

PATCH /faqs/:id

Access	Admin bearer
Purpose	Update FAQ.
Request	Partial create body.
Success data	Faq

DELETE /faqs/:id

Access	Admin bearer
Purpose	Delete FAQ.
Request	Path UUID
Success data	No content

Notes: 204.

11. Subscription plans and payments

GET /payment-plans

Access	Bearer
Purpose	List active plans converted to user currency.
Request	None
Success data	Converted PaymentPlan[]

Notes: Falls back if user has no preferred currency.

GET /payment-plans/raw

Access	Currently public
Purpose	List raw base-currency plans.
Request	None
Success data	PaymentPlan[]

Notes: Comment says admin view, but no guards are applied.

GET /payment-plans/raw/:id

Access	Currently public
Purpose	Get raw plan.
Request	Path UUID
Success data	PaymentPlan

Notes: No guard.

POST /payment-plans

Access	Currently public
Purpose	Create plan.
Request	{ name, description?, durationMonths >=1, basePrice >=0, baseCurrencyId UUID, isActive? }
Success data	PaymentPlan

Notes: Should be admin protected before production.

PATCH /payment-plans/:id

Access	Currently public
Purpose	Update plan.
Request	Partial create body.
Success data	PaymentPlan

Notes: Should be admin protected.

DELETE /payment-plans/:id

Access	Currently public
Purpose	Delete plan.
Request	Path UUID
Success data	Deletion result

Notes: Should be admin protected.

POST /payment/checkout/:planId

Access	Bearer
Purpose	Initiate provider checkout.
Request	Path plan UUID
Success data	Payment plus provider checkout/redirect URL

Notes: User identity comes from bearer token.

POST /payment/:paymentId/verify

Access	Bearer
Purpose	Verify own payment and activate subscription.
Request	Path payment UUID
Success data	Verification/subscription result

GET /payment/webhook/konnect

Access	Provider callback; no runtime guard
Purpose	Process Konnect callback.
Request	Query payment_ref
Success data	Webhook result

Notes: ApiBearerAuth metadata is present but no Firebase guard.

POST /payment/webhook/moyasar

Access	Provider callback; no runtime guard
Purpose	Process Moyasar callback.
Request	Query paymentId
Success data	Webhook result

Notes: Provider signature/authenticity verification should be confirmed.

Critical implementation note

PaymentPlanController imports PaymentPlanService from **./payment-plan.service.spec**, not the production service file. A successful Nest startup/build should be verified after deployment, and this import should be corrected. The plan mutation endpoints also lack Firebase/Admin guards despite being described as admin-style CRUD.

12. Administrator API

GET /admin/users?page=1&limit=25

Access	Admin bearer
Purpose	Paginated user list.
Request	Query page?, limit?
Success data	Pagination object from UsersService

GET /admin/users/:id

Access	Admin bearer
Purpose	Get SQL user.
Request	Path UUID
Success data	User

PATCH /admin/users/:id

Access	Admin bearer
Purpose	Update profile, role, status, subscription.
Request	Partial UpdateUserDto
Success data	User

Notes: Fields: displayName, photoURL, currencyId, currentCountryId, role, subscriptionStatus, isActive.

PATCH /admin/users/:id/role

Access	Admin bearer
Purpose	Promote/demote.
Request	{ role: user|admin }
Success data	User

PATCH /admin/users/:id/active

Access	Admin bearer
Purpose	Activate/deactivate.
Request	{ isActive: boolean }
Success data	User

DELETE /admin/users/:id

Access	Admin bearer
Purpose	Delete SQL user record.
Request	Path UUID
Success data	Deletion result

Notes: Does not delete Firebase identity.

GET /admin/stats/overview

Access	Admin bearer
Purpose	Totals and subscription/admin breakdown.
Request	None
Success data	Overview object

GET /admin/stats/users-per-country

Access	Admin bearer
Purpose	Aggregate users by country.
Request	None
Success data	Aggregation rows

GET /admin/stats/users-per-currency

Access	Admin bearer
Purpose	Aggregate users by currency.
Request	None
Success data	Aggregation rows

GET /admin/stats/users-per-provider

Access	Admin bearer
Purpose	Aggregate by auth provider.
Request	None
Success data	Aggregation rows

GET /admin/stats/signups-over-time

Access	Admin bearer
Purpose	Daily signups.
Request	Query days? default 30
Success data	Time-series rows

GET /admin/stats/revenue

Access	Admin bearer
Purpose	Revenue/payment status summary.
Request	None
Success data	Revenue summary

GET /admin/stats/dashboard

Access	Admin bearer
Purpose	All dashboard aggregates in one call.
Request	None
Success data	Combined stats object

13. Deployment and integration contract

Required environment groups

Area	Typical variables / requirement
Runtime	NODE_ENV=production, PORT=3003 (or chosen internal port)
Database	DB_TYPE=mysql, DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD, DB_NAME
Firebase	Firebase Admin credentials/configuration consumed by FirebaseModule; FCM enabled for push
Translation + speech	TRANSLATOR_SERVICE_URL=http://127.0.0.1:8000; service exposes /ai/translate, /ai/translate-json, /ai/speech-to-text, /ai/text-to-speech
Receipt OCR	OCR_SERVICE_URL and configured receipt endpoint expected by ReceiptExtractionService
Google Places	GOOGLE_MAPS_API_KEY with Places API (New) enabled; key restricted server-side
Mail	SMTP/mailer settings used by password reset
Payments	Konnect/Moyasar credentials and public HTTPS callback URLs

Production verification checklist

- Run npm ci and npm run build; start dist/main.js with PM2 and --update-env.
- Confirm GET /api-docs and a public reference endpoint.
- Confirm Firebase bearer verification with GET /users/me.
- Confirm AI service routes from the VPS: curl http://127.0.0.1:8000/openapi.json.
- Confirm Places API with /explore/nearby and receipt merchant matching.
- Confirm writable /files/profile-images and /files/interpreter directories.
- Use HTTPS for geolocation, microphone, camera, OAuth, push notifications, and payment callbacks.
- Keep AI and database ports private; expose the Nest API only through a reverse proxy/firewall.

Health and smoke-test examples

```
curl https://api.example.com/
curl https://api.example.com/api-docs
curl -H "Authorization: Bearer $TOKEN" https://api.example.com/users/me
curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"latitude":33.88,"longitude":10.10,"radius":3000,"languageCode":"en"}' \
  https://api.example.com/explore/nearby
```

14. Known risks and recommended hardening

Priority	Finding	Recommended action
Critical	Payment plan mutation routes are unguarded.	Add FirebaseAuthGuard + AdminGuard to raw CRUD routes.
Critical	PaymentPlanController imports service from a .spec file.	Import ./payment-plan.service and add startup/e2e coverage.
High	No global ValidationPipe in main.ts.	Install whitelist:true, transform:true, forbidNonWhitelisted as appropriate; retest clients.
High	OAuth redirect URI is hard-coded to localhost.	Move redirect/base URL to environment configuration.
High	Payment webhook authenticity is not evident in controller.	Validate provider signatures/references and make handlers idempotent.
Medium	Receipt list query DTO is advertised but ignored.	Bind @Query() and pass merchant/page/limit to service.
Medium	Nearby guidance is in one-minute testing mode.	Restore one-hour check and longer duplicate suppression before production.
Medium	CORS accepts all origins.	Restrict to production frontend/admin origins.
Medium	No explicit version prefix.	Consider /api/v1 before external integrations depend on paths.
Low	Swagger annotations do not fully model every payload.	Add ApiBody/ApiResponse DTOs for explore, interpreter, statistics and webhooks.

Suggested client rules

- Treat data as the endpoint payload and never assume a raw body outside the standard envelope.
- On 401, refresh once; if tokens are absent or refresh fails, clear session state and redirect to /login.
- Do not retry validation failures. Retry 502/503 with bounded exponential backoff and a visible error modal.
- Use idempotency protections for payment verification and avoid duplicate expense creation after receipt scans.
- Refetch profile after changing language, then reset/refetch translated datasets.
- Open Google Maps using googleMapsUri when available; otherwise construct a coordinates query.

Appendix A - Endpoint index

Auth & user

Method and path
POST /auth/register
POST /auth/login
POST /auth/oauth
POST /auth/firebase-token
POST /auth/refresh
POST /auth/send-reset-mail
POST /auth/validate-reset-mail
POST /auth/reset-password
GET /users/me
PATCH /users/me
DELETE /users/me

Expenses

Method and path
POST /expense-categories
GET /expense-categories
GET /expense-categories/:id
PATCH /expense-categories/:id
DELETE /expense-categories/:id
POST /expenses
POST /expenses/from-receipt
GET /expenses
GET /expenses/statistics
GET /expenses/:id
PATCH /expenses/:id
DELETE /expenses/:id

Receipts

Method and path
POST /receipts/extract
POST /receipts/merchant-match
POST /receipts/receipt-recommendations/:receiptId
GET /receipts
GET /receipts/:id
PATCH /receipts/:id
DELETE /receipts/:id
POST /guest/receipts/extract

Explore & interpreter

Method and path
POST /explore/nearby
POST /explore/check-in
POST /guest/explore/nearby
GET /guest/explore/place/:placeId
POST /guest/explore/place/:placeId/narration
POST /interpreter/conversations
GET /interpreter/conversations
GET /interpreter/conversations/:id
POST /interpreter/conversations/:id/turns
DELETE /interpreter/conversations/:id
POST /guest/interpreter/turn

Communication

Method and path
POST /notifications
POST /notifications/check-country
GET /notifications/me
GET /notifications/:id
PATCH /notifications/:id
PATCH /notifications/:id/read
PATCH /notifications/me/read-all
DELETE /notifications/:id
POST /notifications/devices
DELETE /notifications/devices/:token
GET /chat/conversations/me
GET /chat/conversations/:id/messages
POST /chat/conversations/:id/messages
PATCH /chat/conversations/:id/read
GET /chat/conversations

Reference & translation

Method and path
GET /currencies
GET /currencies/convert
GET /currencies/:id
POST /currencies
PATCH /currencies/:id
DELETE /currencies/:id
GET /countries

Method and path
GET /countries/:id
POST /countries
PATCH /countries/:id
DELETE /countries/:id
GET /languages
GET /languages/search
GET /languages/:id
POST /languages
PATCH /languages/:id
DELETE /languages/:id
POST /translation/text
POST /translation/json
POST /guest/translation/json

Payments & admin

Method and path
GET /payment-plans
GET /payment-plans/raw
GET /payment-plans/raw/:id
POST /payment-plans
PATCH /payment-plans/:id
DELETE /payment-plans/:id
POST /payment/checkout/:planId
POST /payment/:paymentId/verify
GET /payment/webhook/konnect
POST /payment/webhook/moyasar
GET/PATCH/DELETE /admin/users/*
GET /admin/stats/*

End of implementation reference. Regenerate this document whenever controllers, DTOs, response middleware, integration contracts, or authorization guards change.